NEXT.JS 16 • REACT 19 • NVIDIA NIM OS

AI Travel Planner

Complete Architecture, Data Structure, Pros & Cons, Surgical AI Refinement, and Deployment Guide

Project: AI Travel Planner Tech Stack: Next.js 16 + Tailwind v4 + TypeScript Version 1.0 (2026 Edition)

Table of Contents

1. Application Overview & Architecture

1.1 What is AI Travel Planner?

AI Travel Planner is an intelligent travel operating system that transforms a 9-step questionnaire into a personalized, day-by-day itinerary. Built with **Next.js 16 (App Router)**, **React 19**, **TypeScript**, and **Tailwind CSS v4**, the system generates complete travel plans with activity timelines, budget allocations, live weather forecasts, packing lists, emergency contacts, local customs, and Leaflet route maps.

> **Core Philosophy: "AI as a Data Contract"**
>
> Instead of outputting loose markdown text, the AI generator and surgical editor conform to a single strict JSON schema (`TravelItinerary`). This ensures every trip generated is 100% structured, predictable, and losslessly editable.

1.2 Core Architectural Principles

Surgical Refinement Engine: Natural-language commands (e.g., "*Add local izakayas to Night 3 and remove morning hiking*") apply merge-preserving diffs to existing itineraries without regenerating unmentioned days.
Zero-Configuration Guest UX: The core product operates without requiring user registration, databases, or API keys for maps and weather.
Client-First Persistence: Itineraries are saved in versioned `localStorage` (`atp:itineraries:v1`) with automatic quota management.
Pluggable Storage Seam: Public trip sharing defaults to a zero-config file system store (`data/shares/`) and upgrades seamlessly to Supabase cloud database storage when environment variables are set.

1.3 System Execution Pipeline

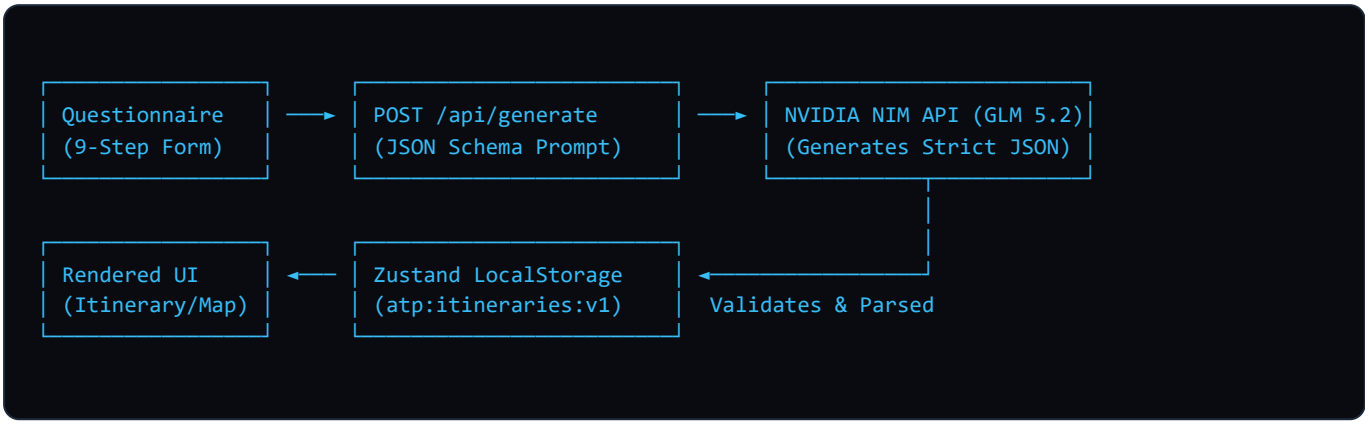

Figure 1: AI Travel Planner End-to-End Execution & Data Flow Pipeline

2. Balanced Assessment: Pros and Cons

2.1 Major Strengths

Zero Configuration Guest UX: Instant usability without requiring sign-up or credit card details.
Surgical Natural Language Edits: Refine trips effortlessly without losing existing daily schedules.
Honest Data Guarantee: Integrates live Open-Meteo weather data; prices are represented as clear estimated ranges.
Modern Developer Experience: Built with Next.js 16 Turbopack, Tailwind CSS v4 tokenized dark glassmorphism, Vitest unit tests, and Playwright E2E suites.

2.2 Operational Limitations & Trade-Offs

Requires NVIDIA API Key for Live Generation: Live itinerary generation requires an active `NVIDIA_API_KEY` (though mock modes run for local testing).
Browser Storage Quotas: Browser `localStorage` has a ~5MB storage ceiling, requiring automated pruning for heavy users.
Public Share Payload Ceiling: Shared trip links enforce a 400 KB payload limit to ensure high performance and prevent system abuse.

2.3 Practical Persona Decision Matrix

Persona	Primary Benefit	Main Challenge	Recommended Workflow
Traveler / End User	Instant complete trip schedule with maps & packing lists	Requires internet for live weather	Use print-optimized Travel Report for offline travel.
Frontend Engineer	Clean React 19 + Tailwind v4 component architecture	Strict TypeScript schema validation	Extend `src/types/itinerary.ts` when adding new trip properties.
DevOps / Architect	Pluggable File ↔ Supabase storage adapter seam	API rate limits on public shares	Enable Supabase service-role adapter for high traffic.

3. Workspace File Structure & Architecture

3.1 Hierarchical File System Tree

```
ai-travel-planner/
├── src/
│   ├── app/                          # Next.js 16 App Router pages & server routes
│   │   ├── api/                      # Serverless backend routes
│   │   │   ├── generate/route.ts     # Main AI generation endpoint (NVIDIA NIM)
│   │   │   ├── refine/route.ts       # Surgical natural language editor
│   │   │   ├── weather/route.ts      # Open-Meteo live weather proxy
│   │   │   ├── share/route.ts        # Public trip link generator
│   │   │   └── chat/route.ts         # AI Travel Assistant endpoint
│   │   ├── budget/page.tsx           # Canonical budget breakdown view
│   │   ├── itinerary/page.tsx        # Main itinerary timeline & day views
│   │   ├── map/page.tsx              # Interactive Leaflet OpenStreetMap view
│   │   ├── plan/page.tsx             # 9-step trip questionnaire form
│   │   ├── profile/page.tsx          # Favorites & trip history
│   │   ├── share/[token]/page.tsx    # Public shared trip viewer
│   │   └── globals.css               # Tailwind v4 glassmorphism styling tokens
│   ├── components/                   # Modular UI components
│   │   ├── ui/                       # Base design system (Button, Card, Badge)
│   │   ├── itinerary/                # Timeline cards, packing list
│   │   ├── budget/                   # Budget breakdown & category sliders
│   │   ├── map/                      # Leaflet map pins & popups
│   │   ├── RefineDrawer.tsx          # Surgical AI editor drawer
│   │   └── AIChatDrawer.tsx          # Conversational travel assistant
│   ├── lib/                          # Utility modules & API wrappers
│   │   ├── nvidia.ts                 # NVIDIA NIM API client wrapper
│   │   ├── weather.ts                # Open-Meteo fetcher & WMO code mapper
│   │   └── storage/                  # Pluggable File ⇔ Supabase storage seam
│   └── types/                        # TypeScript contract specifications
│       ├── itinerary.ts              # TravelItinerary main interface
│       ├── questionnaire.ts          # QuestionnaireData form state
│       └── weather.ts                # Weather forecast interfaces
├── data/
│   └── shares/                       # Zero-config file-based share storage
├── docs/                             # Developer documentation & screenshots
├── public/                           # Brand mark, icons, OG card images
├── scripts/                          # Screenshot & build automation scripts
├── tests/                            # Vitest & Playwright test suites
└── package.json                      # Node dependencies & npm scripts
```

Figure 3: AI Travel Planner Directory Structure & Key Components

4. Data Structure & Schema Specification

4.1 Core TravelItinerary Data Contract

The application is governed by strict TypeScript interface definitions in `src/types/itinerary.ts`:

```
export interface TravelItinerary {
  tripSummary: TripSummary;
  dailyItinerary: DayPlan[];
  weatherForecast?: WeatherDay[];
  accommodations: Accommodation[];
  restaurants: Restaurant[];
  budgetBreakdown: BudgetBreakdown;
  packingChecklist: PackingCategory[];
  transportationDetails: TransportDetail[];
  emergencyContacts: EmergencyContact[];
  hiddenGems: HiddenGem[];
  localCustoms: string[];
  travelTips: string[];
  importantNotes: string[];
}
```

4.2 Sub-Entity Data Models

DayPlan & Activity Interfaces

```
export interface DayPlan {
  day: number;
  date: string;
  title: string;
  summary: string;
  activities: Activity[];
  totalCost: number;
}

export interface Activity {
  time: string;
  endTime: string;
  name: string;
  description: string;
  location: string;
  latitude: number;
  longitude: number;
  duration: string;
  category: ActivityCategory;
  estimatedCost: number;
  tips: string;
}
```

BudgetBreakdown & Accommodation Models

```
export interface BudgetCategory {
  category: string;
  estimated: number;
  percentage: number;
}

export interface BudgetBreakdown {
  totalEstimated: number;
  totalBudget: number;
  currency: string;
  categories: BudgetCategory[];
  savingsTips: string[];
}
```

5. Practical Use-Case Scenarios

5.1 Scenario 1: Initial 7-Day Tokyo Trip Generation

User Input: Destination: "*Tokyo, Japan*" | Budget: "*$2,500*" | Style: "*Culinary & Cultural*".

> **Generated Itinerary Output Summary**
>
> Includes Tsukiji Outer Market breakfast visits, Senso-ji temple morning walks, Shibuya Crossing route maps, Ryokan accommodations, and 6-category budget allocations.

5.2 Scenario 2: Surgical Refinement Command

User Command: *"Add local izakayas to Day 3 evening and swap Day 4 morning to teamLab Planets."*

```
// System Execution:
POST /api/refine
Payload: { itinerary: TravelItinerary, prompt: "Add local izakayas to Day 3..." }
Result: Days 1, 2, 5, 6, 7 remain identical. Days 3 and 4 reflect surgical updates.
```

5.3 Scenario 3: Live Weather Integration

Open-Meteo returns live weather forecasts. If rain is detected, the UI displays weather alerts and auto-suggests indoor museum alternatives.

6. Operational & Deployment Guide

6.1 Quickstart Commands

```
# 1. Install dependencies
npm install

# 2. Configure environment
cp .env.example .env.local

# 3. Launch Turbopack development server
npm run dev

# 4. Execute test suites
npm run test        # Vitest unit tests
npm run test:e2e    # Playwright integration tests
```

6.2 Technical Glossary & Master Index

AI Data Contract

Strict JSON schema enforcement between AI LLM responses and UI rendering components.

Surgical Editing

Modifying specific daily activities using natural language without altering unmentioned days.

Glassmorphism

Modern dark-mode design system with semi-transparent card backdrops and cyan/violet glowing borders.

Storage Adapter Seam

Pluggable interface that seamlessly toggles between local file storage and Supabase database persistence.

A - L

N - Z